Elarion Architecture Guide

Fabric 1.21.1 | Core + Addons | Build and Rebuild Specification

Purpose: a complete, copy-friendly reference for rebuilding your mod ecosystem into one tidy repository.

1. What Elarion is

Elarion is not a collection of independent gameplay mods. It is a platform layer that owns the authoritative state of the world: citizens, nations, titles, visibility rules, chat rules, portal state, jail state, underworld state, and progression gates.

Everything else is an addon that reads or reacts to Core state. Addons do not own identity, render names, or duplicate canonical data.

- Core owns truth.
- Addons interpret truth.
- Render output is derived, never stored as the canonical source.
- Dependencies point toward Core only.

2. Repository layout

Use one Git repository and one Gradle multi-project build. Keep the folder names simple and readable.

```
Elarion/
├─ settings.gradle.kts
├─ build.gradle.kts
├─ gradle.properties
│
├─ platform/
│  └─ core/
│     ├─ build.gradle.kts
│     └─ src/
│
└─ addons/
   ├─ portals/
   ├─ jail/
   ├─ underworld/
   ├─ worlds/
   ├─ chat/
   ├─ tablist/
   ├─ commands/
   ├─ voicechat-hooks/
   ├─ government/
   └─ newspapers/
```

The important rule is not the exact folder names. The rule is that Core lives in one place, and every feature module depends on it.

3. Recommended Fabric settings

- Fabric Loader.
- Fabric API.
- Fabric Language Kotlin if you are comfortable with Kotlin; it reduces boilerplate and makes state-heavy code cleaner.
- Yarn mappings for the development environment.

Use Yarn for day-to-day development. It is the standard choice for Fabric work and is much easier to read than official Mojang names when you are building a large architecture.

Keep Mojang mappings out of the main workflow unless you have a very specific reason. For this project, readability and maintainability matter more than matching official names.

4. What Core owns

- Nation definitions loaded from config.
- Citizen records and player state.
- Titles and status flags.
- Color management through vanilla scoreboard teams.
- Identity rendering for nameplates, tab list, chat, and placeholders.
- Chat channel logic.
- Spawn and world mapping.
- History events and progression records.
- Public API for addons.
- Event bus for addon hooks.

Everything that changes how a player is presented should flow through Core. That includes display name, nation prefix, title, tab order, visibility, and color.

5. Canonical data model

Keep the model small, explicit, and derived where possible.

```
Nation
- id
- displayName
- shortName
- color
- worldId
- spawn
- flags
- portalState
- chatPolicy

Citizen
- uuid
- nationId
- titleId
- nickname
```

```
- status
- joinedAt
- flags

Title
- id
- displayName
- prefix
- suffix
- priority
- visibleUnderUsername

PlayerIdentity (derived)
- displayName
- chatName
- tabName
- prefix
- suffix
- color
- visibilityScope

HistoryEvent
- id
- scope
- actorUuid
- nationId
- message
- category
- timestamp
- metadata
```

6. Status and visibility rules

Do not use many unrelated booleans. Prefer a small state model with clear transitions.

```
CitizenStatus
- ACTIVE
- JAILED
- DEAD
- UNDERWORLD
- EXILED
- DIPLOMAT
```

Status drives gameplay restrictions, rendering, voice chat muting, portal access, and tab visibility.

7. Identity rendering pipeline

Core should compute the player-facing identity from the citizen record, not store the rendered result as truth.

```
Citizen -> Nation -> Title -> Status -> Permission checks -> PlayerIdentity
```

- Nationality controls the base color and prefix.
- Title controls role presentation under the username.

- Status can override normal rendering for jail, underworld, death, or diplomat access.
- Nickname is a cosmetic layer, not the canonical identity.

8. Scoreboard color system

Vanilla scoreboard teams should be treated as the rendering backend for color propagation.

```
Core creates teams such as:
- elarion_black
- elarion_dark_blue
- elarion_dark_green
- elarion_dark_aqua
- elarion_dark_red
- elarion_dark_purple
- elarion_gold
- elarion_gray
- elarion_dark_gray
- elarion_blue
- elarion_green
- elarion_aqua
- elarion_red
- elarion_light_purple
- elarion_yellow
- elarion_white
```

Nation config uses color names like "green" or "gray", and Core maps that value to the correct team. Do not store custom hex colors unless you later build a separate renderer for them.

9. Chat design

For your desired structure, there should be no free global chat channel. Communication is controlled and contextual.

```
Chat channels
- LOCAL: proximity text chat
- NATION: /nc only
- SYSTEM: server and Core notifications
- ADMIN: operator and moderation traffic
```

Local chat should be the default. Nation chat should be the only long-range player conversation channel. That means a player talks locally unless they explicitly use /nc.

```
/nc message
```

Nation chat is visible only to citizens of that nation. Core should apply nation prefix formatting automatically.

```
Example nation chat format:
[%nation_short%] %player% » %message%
```

10. Nicknames and command name resolution

Your old nicknamer functionality should be absorbed into Core as a name resolution service. It should not remain a separate authority.

- Commands should resolve the player by nickname, display name, or username.
- Command suggestions should respect visibility rules.
- Hidden players should not be targetable by ordinary players until portal access allows it.
- The name system must prefer canonical player identity, then nickname, then fallback username.

11. Portals addon design

Portals is where your progression gates, travel rules, and animated portal behavior live.

The portal block should have explicit states. A dormant portal must not look active.

```
PortalState
- DORMANT
- ACTIVE
- OPEN
- LOCKED
- RESTORING
- DISABLED
```

Dormant means the portal is effectively hidden or visually suppressed. The animated texture should be invisible or replaced with a neutral block appearance. No particles, no portal sound, and no travel effect should be active.

Active means the portal is visible and animated, but may still be locked or progression-gated. Open means travel is allowed. Locked means the structure exists but cannot be used yet.

```
Portal visual rules
- DORMANT: invisible or inert appearance
- ACTIVE: animated texture visible
- OPEN: animated texture visible and usable
- LOCKED: visible but not usable
- RESTORING: temporary state with progress feedback
- DISABLED: administratively shut down
```

Portal state should be controllable through commands so events, quest rewards, and server progression can toggle the entire travel network quickly.

```
Suggested portal commands
/e portal status
/e portal dormant <id>
/e portal active <id>
/e portal open <id>
/e portal close <id>
/e portal lock <id>
/e portal unlock <id>
/e portal restore <id>
/e portal disable <id>
```

```
/e portal enable <id>
/e portal set-state <id> <state>
```

If you want event control, this is the right level of command granularity. A single admin command namespace is easier to remember than a wide set of unrelated commands.

12. Portal progression and reward rules

Portals should be able to participate in progression rewards, story events, and temporary server states.

- Portal unlocks can be tied to milestones.
- Portal activation can be time-limited for events.
- A restored portal can announce a History event.
- Portal state can be used as a reward for quests or government decisions.

```
Examples
- Ancient Gate Restored
- Northern Route Opened
- Royal Access Granted
- Diplomatic Corridor Activated
```

13. Jail addon design

Jail is a state and location system. It should never own identity rules; it only changes state that Core understands.

- Jailed players are marked through CitizenStatus.JAILED.
- Jail can move the player to a jail world or jail cell.
- Jail should block portal use, external voice chat, and non-admin movement rules.
- Jail release should restore normal Core state.

```
Suggested jail commands
/e jail send <player> <duration> <reason>
/e jail release <player>
/e jail status <player>
/e jail extend <player> <duration>
/e jail setcell <id>
```

14. Underworld addon design

The underworld is a separate state, not just another world. It is a progression and death-rule layer.

- Underworld access should use CitizenStatus.UNDERWORLD or a dedicated underworld flag.
- Voice chat should be disabled while in the underworld.
- Portal use should be restricted according to your design.
- Respawn logic should route through Core spawn services where possible.

```
Suggested underworld commands
/e underworld send <player>
```

```
/e underworld return <player>
/e underworld status <player>
/e underworld setspawn
```

15. Multiworld integration

Do not let Core manage the low-level world lifecycle. Core should only know which world a nation belongs to and how to resolve that world when needed.

- A separate worlds addon can create, load, and register worlds.
- Core stores only world identifiers and world rules.
- Portals and respawn logic ask Core for the target world, then resolve it through the worlds addon or the server API.
- This keeps the architecture clean and makes world management replaceable later.

16. Simple Voice Chat integration

Voice chat should be treated as a hook layer, not a core dependency. The integration addon listens to Core status and applies voice restrictions.

```
Voice chat rules
- ACTIVE: voice allowed
- JAILED: voice disabled or muted
- UNDERWORLD: voice disabled or muted
- EXILED: follow admin rule
- DIPLOMAT: follows normal visibility rules unless restricted elsewhere
```

The integration should react to state changes rather than polling. When the citizen status changes, the voice restriction should update immediately.

17. Tab list design

The tab list should respect visibility rules and nation progression.

- Before portals open, a player should see only their nation group and permitted admins.
- After portal unlocking, visible player scope can expand according to your design.
- Operators should be shown at the bottom as admins.
- Nation colors should be reflected in the tab list through the scoreboard team system.

The tab list should be a rendering of Core data, not a separate source of truth.

18. Command architecture

Keep commands organized by module, but expose them through one consistent namespace for administration.

```
Suggested root command namespace
/e ...
```

- e portal ...
- e jail ...
- e underworld ...
- e nation ...
- e citizen ...
- e title ...
- e world ...
- e reward ...
- e history ...
- e chat ...

Player-facing commands should stay minimal. Administrative controls should be discoverable, namespaced, and consistent.

19. How your old mods map into the new architecture

```
Old mod          -> New home
Teams            -> Core scoreboard color system
Namer            -> Core identity resolution
Title renderer   -> Core title/identity rendering
Jail             -> addons/jail
Multiworld       -> addons/worlds
Nation chat      -> Core chat system
Portals          -> addons/portals
Underworld       -> addons/underworld
Commands disabler-> Core commands policy or addons/commands
Voice chat hook  -> addons/voicechat-hooks
Tab list         -> Core rendering + addons/tablist if needed
```

20. Best order to build it

- Phase 1: Core only - nations, citizens, titles, colors, chat, placeholders, storage.
- Phase 2: Worlds addon - world registration and resolution.
- Phase 3: Portals addon - dormant, active, open, lock, unlock, restore.
- Phase 4: Jail addon - prisoner flow and restrictions.
- Phase 5: Underworld addon - death realm flow and respawn rules.
- Phase 6: Voice chat hooks - mute and restrict based on Core state.
- Phase 7: Tab list and command policy - visibility polish.
- Phase 8: Government, newspapers, history, and progression rewards.

21. Non-negotiable design rules

- Core is the only canonical state owner.
- Addons never store duplicate truth.
- Identity is derived, not manually formatted in multiple places.
- No addon should directly override another addon's responsibilities.
- The scoreboard team system is an implementation detail for color propagation.

- Commands should be consistent and namespaced.
- Visibility rules must be enforced before tab list, chat, command suggestions, and teleport targets are exposed.

22. Where to start

For a first build, start small enough that the project becomes real quickly.

```
Minimum first milestone
1. Core loads config and player files.
2. Nations exist in YAML.
3. Citizens join a nation.
4. Scoreboard colors apply correctly.
5. /nc works.
6. Tab list shows the correct visible players.
7. Titles render under the username.
8. Portal state can be toggled from dormant to active.
```

After that, add portals, then jail, then underworld, then voice chat hooks. Do not start with every module at once.

23. Final summary

The architecture you want is a platform core with tightly controlled addons. Core owns identity, nations, titles, colors, chat, placeholders, and state. Portals, jail, worlds, underworld, voice chat hooks, government, and newspapers are all consumers of that state. This gives you one authoritative rules engine and a tidy modpack that is easy to reason about and easy to extend.